

Roteiro de Laboratório 2 — Escalonamento e sections

Unidade 03 | Continuação do Laboratório 1. Entrega pelo Moodle ao final da aula.

Objetivos

- Entender os efeitos das diretivas de escalonamento (`static`, `dynamic`, `guided`) na paralelização de laços.
- Medir esse efeito num problema real, onde o custo de cada iteração não é o mesmo.
- Diferenciar `parallel for` (divide iterações de um laço) de `sections` (divide blocos de código distintos).

Preparação

```
cd codigo
```

Os fontes deste laboratório estão soltos em `codigo/`, sem `Makefile`: compile e rode cada um com `gcc` conforme os comandos de cada parte.

Parte 1 — Como cada política distribui as iterações

`escalonamento_demo.c` roda um laço de 16 iterações com 4 threads, imprimindo qual thread processou cada iteração `i`. Só uma linha de `#pragma` fica ativa por vez.

1. Compile e rode com `schedule(static)` (a que já vem ativa):

```
gcc -O2 -Wall -fopenmp -o escalonamento_demo escalonamento_demo.c
./escalonamento_demo
```

2. Comente essa linha, descomente `schedule(static, 2)`, recompile e rode de novo. Repita para `schedule(dynamic)`, `schedule(dynamic, 2)` e `schedule(guided)`.
3. Pesquise a definição exata de `static`, `dynamic` e `guided` no OpenMP, e o papel do número que aparece depois da vírgula (o *chunk*).
4. **Responda:**
 - Para cada política testada, o padrão de distribuição observado bateu com a definição pesquisada?
 - O que muda ao acrescentar um inteiro à cláusula, como em `dynamic, 2`? Em que situação isso pode ser vantajoso?

Parte 2 — `static` não escala: descobrindo por quê

`conta_primos.c` é o mesmo programa do Laboratório 1: `conta_paralela` já vem com `#pragma omp parallel for reduction(+:total)` pronto, exatamente como você escreveu lá. A única diferença é que o `pragma` **não tem cláusula `schedule`** — e é isso que este roteiro pede para investigar. (A versão sequencial não entra aqui: ela não muda com o `schedule`, então o que importa é comparar o tempo paralelo entre políticas, não contra ela.)

1. Acrescente `schedule(static)` ao `pragma` (deixando explícito o que já era o padrão), compile e monte uma tabela de tempo variando threads:

```
gcc -O2 -Wall -fopenmp -o conta_primos conta_primos.c
OMP_NUM_THREADS=4 ./conta_primos
```

Threads	Tempo com <code>static</code> (s)	Tempo com <code>dynamic</code> (s)
---------	-----------------------------------	------------------------------------

2. Troque **só** a cláusula para `schedule(dynamic)`, recompile e complete a mesma tabela.

3. **Responda:** o tempo de `static` continua caindo proporcionalmente ao dobrar as threads, ou ele emperra a partir de algum ponto? E o de `dynamic`?
4. **Responda, explicando a causa:** `eh_primo(n)` faz um laço até `sqrt(n)` — testar se 9999999 é primo custa muito mais do que testar se 3 é primo. Com `schedule(static)` sem `chunk`, o intervalo `[2, limite)` é dividido em **um bloco contíguo por thread**. Qual thread recebe o bloco dos números maiores? O que isso significa para o tempo total, já que o programa só termina quando a **última** thread termina?
5. Experimente `schedule(static, 1000)` (blocos pequenos, em rodízio) e `schedule(dynamic, 1000)`. Algum se aproxima do melhor tempo sem pagar o custo de coordenação do `dynamic` sem `chunk`?

Parte 3 — sections: tarefas diferentes ao mesmo tempo

Até aqui, todo paralelismo veio de dividir as iterações de **um** laço. A construção `sections` divide um bloco de código em pedaços diferentes, cada um executado por uma thread — não iterações de um laço, e sim trechos de código distintos.

`sections.c` abre uma região paralela com 4 threads e, dentro dela, divide o trabalho em 4 seções.

1. Compile e rode como está:

```
gcc -O2 -Wall -fopenmp -o sections sections.c
./sections
```

2. **Responda:** qual thread executou cada seção? A ordem das seções no código-fonte determina a ordem de execução?
3. Descomente as quatro chamadas a `sleep(1)` (uma em cada seção), recompile e rode de novo, cronometrando:

```
gcc -O2 -Wall -fopenmp -o sections sections.c
time ./sections
```

4. **Responda:** o tempo total ficou perto de 4 segundos (a soma dos `sleep`) ou perto de 1 segundo (o maior valor entre eles)? Por que o modelo de execução das `sections` produz esse resultado?

Entrega

- As linhas de `schedule` testadas na Parte 1 e as respostas do item 4.
- O `conta_primos.c` final e a tabela de tempos da Parte 2.
- As respostas da Parte 3.